StormWatch Live

A research toolkit for forecasting the erratic, terrain-driven winds that drive explosive wildfire behavior — and a live hazard map that puts the science in front of people.

The core idea, in one sentence: use **synoptic models** to predict *when* dangerous winds arrive, then use **terrain downscaling** (WindNinja) to predict *where* on the landscape they'll be worst — but only for the events where that physics is actually valid. Knowing *when not to trust the model* is half the work.

Why this exists

Operational wind forecasts run at ~3 km. The winds that kill — canyon jets, downslope windstorms, lee acceleration — live at sub-kilometer scales that 3 km grids smear out. This project learns from **22 historical extreme fire-wind events** (Camp, Tubbs, Thomas, Marshall, Yarnell Hill, Lahaina, back to Mann Gulch 1949) to answer two questions for any new event:

1. **Which physical mechanism made the wind?** Not every dangerous wind is downscalable. A canyon jet is; a thunderstorm downburst or a fire's own plume-driven indraft is not.
2. **Given the mechanism, how much can we trust a terrain-downscaled forecast, and which axis (timing / speed / direction) is most likely to bust?**

> What determines whether an event is usable is not how famous or how erratic it was — it's *which physical process* produced the wind.

The four-mechanism model

Every event is sorted into one of four physical mechanisms, each with a different WindNinja trust level:

Mechanism	What it is	WindNinja applicability
SYNOPTIC_TERRAIN	Strong gradient flow channeled/accelerated by terrain (gap winds, downslope windstorms, lee waves)	**HIGH** — the core use case
PBL_TRANSIENT	A transition: cold-front wind shift, or low-level-jet decoupling then morning mix-down	**PARTIAL** — snapshots before/after, not the transition itself
CONVECTIVE_OUTFLOW	Thunderstorm downburst / derecho gust front	**NONE** — steady-state assumption invalid
FIRE_GENERATED	Pyroconvection, plume collapse, indrafts — the fire makes its own wind	**NONE** — needs a coupled fire-atmosphere model

The classifier is **rule-based, not ML** — N is tiny, the physics is known, and you need to *trust and explain* each bin. It scores only over the evidence actually present and reports an `evidence_fraction`, so a confident label built on 2 of 8 diagnostics is flagged as low-confidence rather than hidden.

Repository map

Path	What it is
`mechanism_classifier.py`	The four-mechanism classifier + calibratable `THRESHOLDS`. Start here.
`live_forecast.py`	**Live fire-wind outlook** — feeds *current* forecast data through the classifier (see below).
`bc_label_generator.py`	Inner loop: sweeps boundary conditions through WindNinja, scores vs. observed RAWS, finds the optimal BC.
`bc_outer_trainer.py`	Outer loop: leave-one-event-out CV learning HRRR → BC-correction mapping.
`confidence_field.py`	Per-station confidence labels from a 25-member WindNinja ensemble.
`hindcast_event_library.md`	The 22-event catalogue with mechanism tags and station anchors.
`weather-alerts.html`	The StormWatch Live web app (NWS alerts, SPC outlooks, fire weather, hazard layers).
`mcp-server/`	Node.js MCP server (v5) exposing ~14 weather/hazard tools (alerts, SPC, fire weather, gauges, tropical, AQI, aviation, earthquakes, compound briefings).
`reconstruction_case*.md`	Per-event scientific write-ups.
`CLAUDE_CODE_HANDOFF.md`	Detailed developer handoff for the analysis backbone.

Live fire-wind outlook (`live_forecast.py`)

Points the project's own classifier at the **next 24–48 h** instead of a past event. Pure standard library — no `pip`, no `conda`, runs under system Python.

> 📊 **Today's outlook:** `FIRE_WIND_OUTLOOK.md` — auto-updated daily by a GitHub Action. Also live as the **Outlook** tab in the web app, and as machine-readable `data/live_fire_wind.json`.

```
python live_forecast.py                    # all catalogued fire sites
python live_forecast.py "Yarnell AZ"     # any named place
python live_forecast.py 39.76,-121.37    # raw lat,lon
python live_forecast.py --json           # machine-readable (used by the daily Action)
python live_forecast.py --hours 36 "Paradise CA"
```

Example (CRITICAL day at Yarnell Hill, AZ):

```
=== [!!] Yarnell, Arizona ===
  threat    : CRITICAL
  mechanism : SYNOPTIC_TERRAIN
  WindNinja : HIGH  - core use case; downscale the sustained flow
  peak wind : 25.2 mph sustained, gust 37.6 mph from 227deg
  driest    : RH min 6%
  drivers   : 700hPa max 20.48 m/s | CAPE 0.0 | lapse 8.36 C/km | no convection
```

Honest by design. Diagnostics a point forecast genuinely can't see (satellite plume, sub-hourly downburst blast, terrain cross-ridge component, frontal thermodynamics vs. the diurnal cycle) are left unevaluated, not guessed. A **fire-weather threat gate** (peak gust + minimum RH) means a benign day is reported as benign instead of being forced into a scary mechanism label, and the headline mechanism is chosen only from those clearing the classifier's own `min_evidence_fraction`.

Data source. `live_forecast.py` uses the free Open-Meteo model (surface + 700 hPa) as a stand-in to prove the logic live. The high-resolution terrain path — **Herbie → HRRR → WindNinja** — is the separate, heavier seam described in `CLAUDE_CODE_HANDOFF.md` and is what delivers the actual sub-kilometer skill.

Status & roadmap

- ✅ Classifier, BC label generator, BC trainer, confidence field — logic complete and self-tested
- ✅ Live classification seam (`live_forecast.py`) on a free data source
- ⏳ Wire the high-res seams: Herbie HRRR retrieval, WindNinja solver calls
- ⏳ Timing-bust detector (ensemble arrival-uncertainty)

Tech stack

Python (analysis) · HTML/JavaScript (web app) · Node.js (MCP server). Live data: Open-Meteo, NWS/api.weather.gov, SPC, NHC, USGS.

Disclaimer

Research and educational tool. **Not an official forecast.** For life-safety decisions, always rely on the National Weather Service and local authorities.

License
